

## 1. First Palindromic String in an Array and find its complexity

### Aim

To write a Python program to find the **first palindromic string in an array** and analyze its time and space complexity.

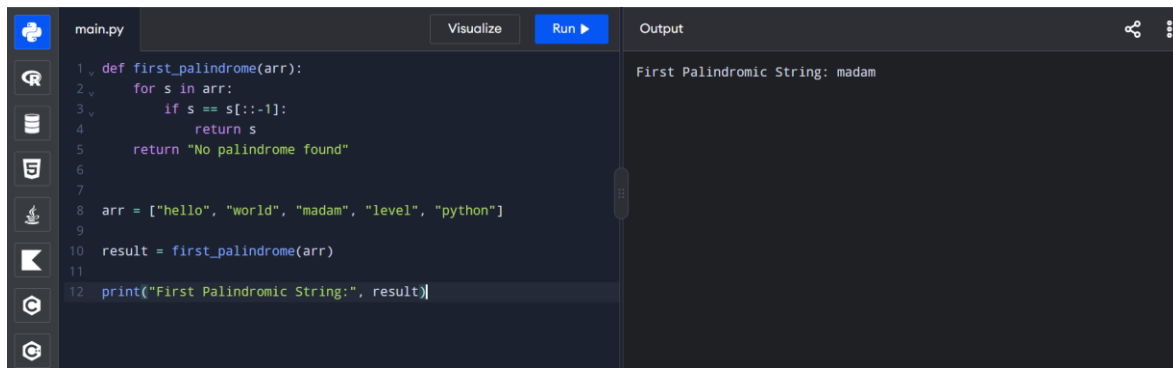
### Algorithm

1. Start.
2. Read the array of strings.
3. Traverse the strings one by one.
4. For each string, reverse it and compare it with the original string.
5. If both are equal, print the string as the first palindrome and stop.
6. If no palindrome is found, print "No palindrome found".
7. Stop.

### Source Code

```
def first_palindrome(arr):  
    for s in arr:  
        if s == s[::-1]:  
            return s  
    return "No palindrome found"  
  
arr = ["hello", "world", "madam", "level", "python"]  
result = first_palindrome(arr)  
print("First Palindromic String:", result)
```

## Output



```
main.py Visualize Run ▶ Output
1 def first_palindrome(arr):
2     for s in arr:
3         if s == s[::-1]:
4             return s
5     return "No palindrome found"
6
7
8 arr = ["hello", "world", "madam", "level", "python"]
9
10 result = first_palindrome(arr)
11
12 print("First Palindromic String:", result)
```

First Palindromic String: madam

## Complexity

- **Best Case:**  $O(m)$
- **Worst Case:**  $O(n \times m)$
- **Space Complexity:**  $O(m)$

Where **n** = number of strings and **m** = length of a string.

## Result

Thus, the Python program to find the **first palindromic string in an array** was successfully implemented and executed. The first palindromic string was "**madam**". The time complexity is  **$O(n \times m)$**  in the worst case and the space complexity is  **$O(m)$** .

## 2. Common Elements Between Two Arrays and find its completeness

### Aim

To write a Python program to find the **common elements between two arrays** and analyze its time and space complexity.

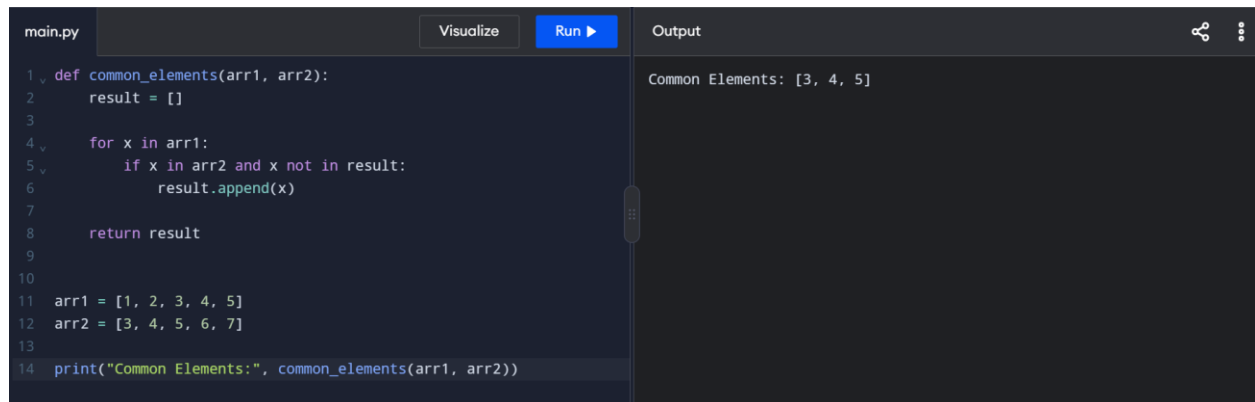
### Algorithm

1. Start.
2. Read two arrays.
3. Compare each element of the first array with elements of the second array.
4. If an element is present in both arrays, add it to the result.
5. Print the common elements.
6. Stop.

### Source Code

```
def common_elements(arr1, arr2):  
  
    result = []  
  
    for x in arr1:  
  
        if x in arr2 and x not in result:  
  
            result.append(x)  
  
  
    return result  
  
arr1 = [1, 2, 3, 4, 5]  
  
arr2 = [3, 4, 5, 6, 7]  
  
print("Common Elements:", common_elements(arr1, arr2))
```

## Output

A screenshot of a code editor interface. The left pane, titled 'main.py', contains a Python script. The script defines a function 'common\_elements' that takes two arrays, 'arr1' and 'arr2', and returns a list of common elements. It uses a 'for' loop to iterate through 'arr1' and an 'if' statement to check for elements in 'arr2' that are not already in the 'result' list. Below the function, two arrays are defined: 'arr1 = [1, 2, 3, 4, 5]' and 'arr2 = [3, 4, 5, 6, 7]'. A 'print' statement at the bottom calls the function and prints the result. The right pane, titled 'Output', shows the result of running the code: 'Common Elements: [3, 4, 5]'. The interface includes a 'Visualize' button and a 'Run' button with a play icon.

## Complexity

Let:

- $n$  = number of elements in the first array
- $m$  = number of elements in the second array

**Time Complexity:**  $O(n \times m)$  in the worst case.

**Space Complexity:**  $O(k)$ , where  $k$  is the number of common elements.

## Result

Thus, the Python program successfully finds the **common elements between two arrays**. The worst-case time complexity is  **$O(n \times m)$** .

### 3. Bubble Sort with Big-O Analysis

#### Aim

To write a Python program to implement **Bubble Sort** and analyze its **Big-O time complexity**.

#### Algorithm

1. Start.
2. Read an array of elements.
3. Compare adjacent elements.
4. If the first element is greater than the second, swap them.
5. Repeat the process for all elements.
6. After each pass, the largest unsorted element moves to the end.
7. Repeat until the array is sorted.
8. Print the sorted array.
9. Stop.

#### Source Code

```
def bubble_sort(arr):  
  
    n = len(arr)  
  
    for i in range(n):  
  
        for j in range(0, n - i - 1):  
  
            if arr[j] > arr[j + 1]:  
  
                arr[j], arr[j + 1] = arr[j + 1], arr[j]  
  
    return arr  
  
arr = [5, 3, 8, 4, 2]  
  
print("Sorted Array:", bubble_sort(arr))
```

## Output



The screenshot shows a code editor with a file named 'main.py'. The code implements a bubble sort function. The editor has a 'Visualize' button and a 'Run' button. The output panel on the right shows the result of running the code.

```
1 def bubble_sort(arr):
2     n = len(arr)
3
4     for i in range(n):
5         for j in range(0, n - i - 1):
6             if arr[j] > arr[j + 1]:
7                 arr[j], arr[j + 1] = arr[j + 1], arr[j]
8
9     return arr
10
11
12 arr = [5, 3, 8, 4, 2]
13
14 print("Sorted Array:", bubble_sort(arr))
```

Output: Sorted Array: [2, 3, 4, 5, 8]

## Big-O Analysis

- Best Case:  $O(n^2)$
- Average Case:  $O(n^2)$
- Worst Case:  $O(n^2)$
- Space Complexity:  $O(1)$

## Result

Thus, the Bubble Sort program was successfully implemented. The worst-case time complexity is  $O(n^2)$  and the space complexity is  $O(1)$ .

## 4. Binary Search with Big-O Analysis

### Aim

To write a Python program to implement **Binary Search** and analyze its **Big-O time complexity**.

### Algorithm

1. Start.
2. Take a **sorted array**.
3. Set `low = 0` and `high = n - 1`.
4. Find the middle element.
5. If the middle element equals the target, return its position.
6. If the target is smaller, search the left half.
7. If the target is larger, search the right half.
8. Repeat until the element is found or the search range becomes empty.
9. Stop.

### Source Code

```
def binary_search(arr, target):
```

```
    low = 0
```

```
    high = len(arr) - 1
```

```
    while low <= high:
```

```
        mid = (low + high) // 2
```

```
        if arr[mid] == target:
```

```
            return mid
```

```
        elif arr[mid] < target:
```

```
            low = mid + 1
```

```
        else:
```

```
            high = mid - 1
```

```
    return -1
```

```
arr = [10, 20, 30, 40, 50, 60, 70]
```

```
target = 40

result = binary_search(arr, target)

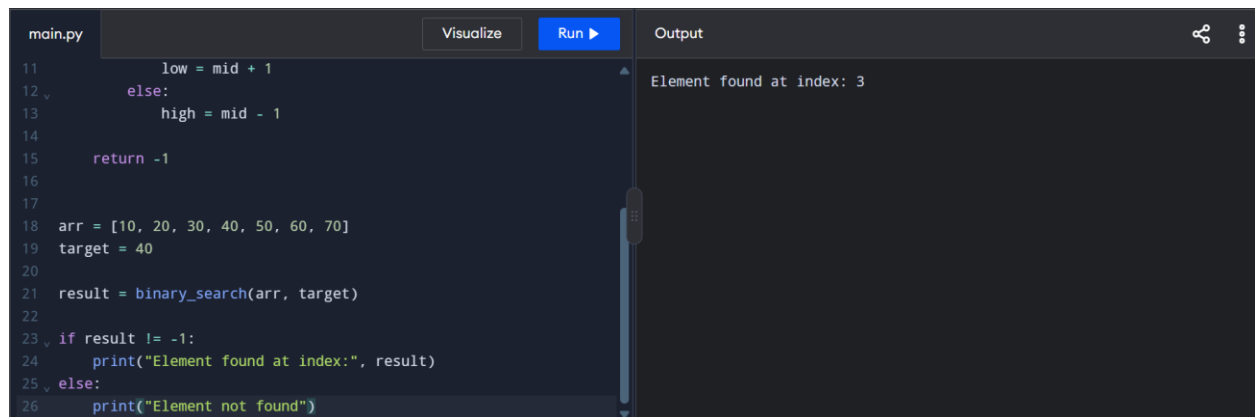
if result != -1:

    print("Element found at index:", result)

else:

    print("Element not found")
```

## Output

A screenshot of a code editor interface. The left pane shows a Python file named 'main.py' with the following code: 

```
11     low = mid + 1
12     else:
13         high = mid - 1
14
15     return -1
16
17
18 arr = [10, 20, 30, 40, 50, 60, 70]
19 target = 40
20
21 result = binary_search(arr, target)
22
23 if result != -1:
24     print("Element found at index:", result)
25 else:
26     print("Element not found")
```

 The right pane, titled 'Output', displays the result of running the code: 'Element found at index: 3'. The interface includes a 'Visualize' button and a 'Run' button with a play icon.

## Big-O Analysis

- **Best Case:**  $O(1)$
- **Average Case:**  $O(\log n)$
- **Worst Case:**  $O(\log n)$
- **Space Complexity:**  $O(1)$

## Result

Thus, the Binary Search program was successfully implemented. The worst-case time complexity is  $O(\log n)$  and the space complexity is  $O(1)$ .